Ace the Exam Series®

FIRST EDITION

2022

DCA

DOCKER CERTIFIED ASSOCIATE

EXAM CRAM NOTES

UP TO DATE EXAM BLUEPRINT
LATEST EXAM QUESTIONS AND REGULARLY REFRESHED CONTENT FOR YOU TO MASTER YOUR EXAM CERTIFICATION.

ACE EXAMS WITH CONFIDENCE:
OUR PRECISE AND COMPREHENSIVE STUDY MATERIAL ENSURES PASSING GRADES.

IPSpecialist
Sharpen Your Skills for the Digital Future

Docker Certified Associate (DCA)

Exam Cram Notes
First Edition

Chapter 01: Introduction

Course Introduction

Docker is powerful container management and execution technology. It is currently the industry-leading container runtime platform, with many features centered on container management and orchestration.

This course is designed to help students prepare for the Docker Certified Associate certification test. Finally, the certification test attempts to certify a Docker practitioner's abilities. We will go over the principles and the goals that you will need to use Docker effectively in this course. Learners will also discover how to use both the essential capabilities of Docker Community Edition (DCE) and the advanced features of Docker Enterprise Edition (DEE).

Developers have switched from application binaries to container images as their deployment artifacts, and they now need to construct container-based applications as part of their new development workflow. This Docker course will teach you how to use the Containers as a Service (CaaS) platform for management and administrative activities.

The book introduces you to the fundamental ideas of containers and microservices. After that, you will learn various orchestration methodologies and environments and the Docker Enterprise platform. The course will show you how to deploy safe, production-ready container-based apps in Docker Enterprise environments as you progress through the book. Later, you will go through each Docker Enterprise component and learn everything there is to know about CaaS management. You will come across important exam-specific themes throughout the book, example questions, and extensive answers to help you prepare efficiently for the exam.

You will learn how to efficiently deploy and maintain container-based setups in production by the end of this Docker containers book, and you will get the skills and knowledge you need to pass the DCA test.

Topics in DCA Certification

Docker awards the DCA Certification (proctored by Examity) to demonstrate your familiarity and proficiency with Docker application deployment. You must

have a working grasp of Docker Enterprise Edition and Docker Swarm to pass the exam and a good understanding of Container Orchestration. You would demonstrate that you can install and configure containerized apps using Docker if you pass this test, making it a good benchmark of your IT skills.

Following are a few critical ideas that you should practice while studying for a DCA certification, in order of their importance:

Container Orchestration

This section focuses on Docker and Container Orchestration fundamentals, and it accounts for around a quarter of your total DCA exam score. It would be best if you also learned how to use various Container Orchestration Tools to help automate the process of container management.

Docker Swarm/Kubernetes

Container orchestration platforms like Kubernetes and Docker Swarm are popular. Kubernetes is an open-source and modular container orchestration platform that provides an efficient container orchestration solution for high-demand applications with complicated configurations. On the other hand, Docker Swarm focuses on simplicity, making it ideal for simple applications that are quick to install and manage. You must have a working grasp of both of these tools and be aware of their most relevant use-cases in various settings when you study for the DCA exam.

Image Creation, Registry, and Management

In a DCA test, Image Creation, Management, and Registry account for around 20% of your entire score. Images, which are the building elements of containerized programs, are the foundation of all Docker containers. An image is an executable package that contains all of the components you will need to run your app.

Installation and Configuration

This is regarded as the most important aspect of the entire DCA education. Although Installation and Configuration represent 15% of your total score, it should be highlighted that in the actual world, a complete understanding of these principles would come in helpful virtually every time.

Security and Networking

Networking and security each account for 15% of the overall score. Docker networking entails utilizing Network Drivers to connect containers.

Authentication, encryption, and transport layer security are all covered in the

DCA security chapter.

Storage and Volume

This chapter accounts for roughly 10% of your total exam score. In Docker, volumes provide a mechanism to store data.

Docker Enterprise Edition

Docker Enterprise Edition (EE) is designed for mission-critical deployments of applications. This fully managed solution includes comprehensive container management, security scanning, and application logging and monitoring. All major Server operating systems can run this version, including Red Hat Enterprise Linux (RHEL), Ubuntu, Oracle Linux, Windows Server 2016, and SUSE Linux Enterprise Server (SLES). It is also compatible with major cloud providers such as Azure and AWS.

Course Features and Tools

This section will introduce you to some of the tools and features available to help you on your journey as you study this course for the Docker Certified Associate.

Cloud Playground

You can access that up here at the top by clicking on this Playground button. The cloud playground will allow you to spin up your servers in the cloud, and you can use those servers to follow along with the lessons as we are going through them and in some of those lessons.

Hands-on Labs

You can also check out these hands-on labs that provide you with a way to work with some of these concepts. The hands-on labs are a more self-contained environment. They will spin up some temporary servers specifically for that lab. They will give you a scenario to complete and some instructions.

Therefore, those hands-on labs are another great way to get hands-on with these actual Docker concepts, which is a great way to learn.

Chapter 02: Modern Infrastructure and Applications with Docker

Introduction

Microservices and containers are two of the most commonly used terms in recent years. We still hear about them these days at conferences worldwide. Although both phrases are used interchangeably when discussing modern applications, they are not interchangeable. In fact, we can run massive monolithic programs in containers and execute microservices without them. When we talk about containers, a well-known word comes to mind: Docker.

- Understanding how applications have changed over time
- Infrastructures
- Processes
- Processes and microservices
- What are containers, exactly
- Understanding the fundamental ideas of containers
- Components for Docker
- Workflows can be built, shipped, and run
- Containers for Windows
- Docker Customization
- Docker's safety

Technical Requirements

Various Docker Engine principles will be covered in this chapter. At the end of this chapter, we will provide several experiments to assist you in comprehending and learning about the principles presented. These experiments can be conducted on your laptop or PC using the Vagrant standalone environment or any Docker host that you have already installed.

Understanding How Applications Have Changed Over Time

The concept of microservices is critical in constructing new modern apps, as we will likely read about it in every IT medium. Let us look back in time to observe how applications have evolved.

Monolithic applications are programs that consolidate all of their components into a single program that runs on a single platform. These applications were not created with reusability or modularity in mind. This meant that whenever a code

component needed to be updated, all of their apps had to be included in the process. For example, all application codes had to be recompiled to work. Things were not as rigid back then, of course.

In terms of responsibilities and features, apps grew in number, with some of these activities being delegated to other systems or even smaller applications. However, the basic components were left unchanged. We chose this programming paradigm since it was easier to run all application components on the same host rather than relying on information from other hosts. Though, network speed was insufficient in this regard. These programs were extremely difficult to grow and improve. In fact, several applications were limited to specific hardware and operating systems, requiring developers to use the same hardware architecture throughout the development process.

Microservices architecture is designed to be stateless. This means that the state of the microservice should be handled outside of its logic since, for example, we need to be able to run many replicas for microservice (scale up or down) and run its content on all nodes of our environment, as required by our global load.

Microservices have the following characteristics:

- We can replace a component with a newer version or even a whole new feature without compromising application functionality when we manage an application in pieces
- Developers can concentrate on a single application feature or function, and they only need to understand how to communicate with other similar components
- Standard HTTP/HTTPS API Representational State Transfer (REST) calls are typically used to connect microservices. The goal of RESTful systems is to improve performance, reliability, and scalability
- Microservices are components designed to have their life cycles. This means that a single harmful component will not significantly impact app usage. Each component will be resilient, and applications will not experience complete outages
- Each microservice can be created in a variety of programming languages, allowing us to select the most performant and portable option
- Let’s look at the concept of modern apps now that we have gone through some of the well-known application designs that have evolved

The following are characteristics of a modern application:

- Microservices will be used to build the components
- The health of the application component will be determined by its resilience
- The states of the component will be controlled externally
- It is going to run all over the place
- It will be set up to allow for simple component updates
- Each application component will be able to function independently but will also provide a method for other components to consume it

Infrastructures

We need to provide some aligned infrastructure architecture for every defined application model developers use for their applications.

All application capabilities run together in monolithic apps, as we have seen. Applications were created for a certain architecture, operating system, libraries, binary versions, and so on in some circumstances. This means we will require at least one hardware node for production and the same node architecture and resources for development. When we factor in earlier contexts, such as certification or preproduction for performance testing, the number of nodes for each application becomes critical in terms of physical space, resources, and money spent on the application.

Developers often require a full production-like environment for each application release, with minimal configuration differences across environments. This is difficult because modifications to an operating system component or functionality must be reproduced across all application contexts. There are numerous technologies available to assist us with these duties, but they are not simple, and the cost of having almost identical surroundings is something to consider. On the other hand, node provisioning could take months because a new application release often necessitates the purchase of new hardware.

Processes

A process is a technique of interacting with an operating system from the ground up. A program is a set of coded instructions that our system will execute; a process is that code in action. It will use system resources such as CPU and memory during process execution. While it will run in its environment, it can share information with another process running in parallel on the same system. Operating systems give tools for manipulating the behavior of this process while it is running.

Understanding the Fundamental Ideas of Containers

It is important to understand the fundamental notions at work regarding containers. Let us break down the container concept into its component parts and try to comprehend each one separately.

Container Runtime

The software and operating system components enabling process execution and isolation will be the runtime for executing containers.

Docker, Inc. offers a container runtime called Docker, built on open-source initiatives sponsored by the company and other well-known companies that support container movement (such as Red Hat/IBM and Google). Other components and utilities are included with this container runtime. In the Docker components section, we will go through each one in-depth.

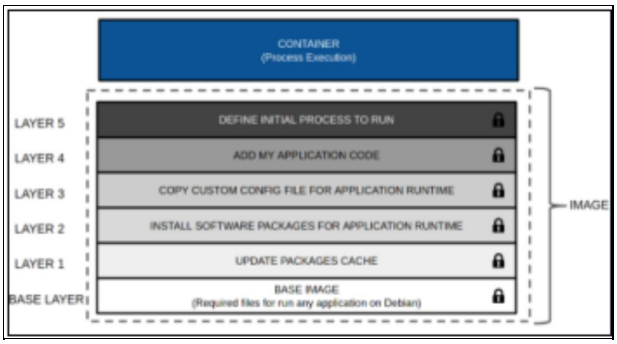
For container creation, we use photographs as templates. All of the information needed for our process or processes to execute smoothly will be contained in images. These components can include binaries, libraries, configuration files, and other parts of the operating system or components you have created specifically for this application.

Images

Images, like templates, cannot be changed. This means they do not alter from one execution to the next. We will obtain the same results every time we use an image. To govern the behavior of distinct processes between contexts, we will merely alter configuration and environment. Developers will construct their application component template, certain that if the application passes all of its tests, it will perform as intended in production. These features ensure that workflows are faster and less time to market.

Containers

A container is a process with all of its prerequisites that operates independently of all other processes on the same host. We can state that containers are formed using images as templates now that we understand what templates are. In reality, a container adds a new read-write layer on top of image layers to store filesystem changes from these layers. The different stages involved in container execution are depicted in the diagram below. As it can be seen, the top layer is referred to as the container because it is read-write and allows modifications to be saved on the host disk:



Process Isolation

A kernel provides namespaces for process isolation, as previously discussed. Let us take a look at what each namespace has to offer. For the following, each container has its kernel namespace:

Processes: The containers' main process will be the parent of all other processes

Network: Each container will have its own network stack, including interfaces and IP addresses, and will make use of host interfaces

Users: We will be able to link distinct host user IDs to container user IDs

IPC: Each container will have its own shared memory, semaphores, and message queues, which will prevent other processes on the host from interfering

Mounts: Each container will have its own root filesystem

UTS: Each container will have its own hostname, and the hosts' time will be synchronized

Docker daemon

Although it can run as a standalone process, the Docker daemon is commonly run as a systemd-managed service (it is very useful when debugging daemon errors, for example). **The dockerd**, as previously mentioned, provides an API interface through which clients can submit commands and interact with the daemon. Containers are managed via **containerd**. It was introduced as a distinct daemon in Docker 1.11 and is in charge of storage, networking, and namespace interaction. It also handles image shipping before running containers with the help of an additional component. RunC, an external component, is the container's true executor. Its function just receives a command to start a container. Since the community shares these components, Docker solely supplies **dockerd**.

Docker Client

A Docker client is a program that allows you to communicate with a server. To

conduct any operation, such as generating an image or launching a container, it must be connected to a Docker daemon.

We can administer a connected distant Docker daemon or run a Docker daemon and client on the same host machine. A server-side REST API is used to interact between the Docker client and daemon. As we mentioned before, this communication can occur over UNIX sockets (by default) or a network interface.

Docker Objects

Using the Docker client command line, the Docker daemon will manage all types of Docker objects:

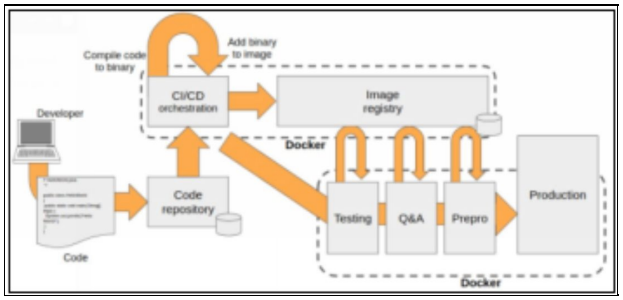
- IMAGE
- CONTAINER
- VOLUME
- NETWORK
- PLUGIN

Other things become available only when Docker Swarm orchestration is deployed:

- NODE
- SERVICE
- SECRET
- CONFIG
- STACK
- SWARM

Workflows can be Built, Shipped, and Run

Docker provides tools for creating images (container templates), distributing those images to systems other than the one used to create the image, and eventually executing containers based on those images:



Docker Engine will be involved in all workflow steps, and we can use one or

more hosts, including our developers' laptops, during these operations.

Building

It is simple to create applications with containers. The standard procedure is as follows:

- An application is normally coded on the developer's computer
- A commit is deployed when the code is ready, or a new release, new functionalities, or a bug has simply been corrected
- If our code has to be compiled, we can do it now. If we are writing code in an interpreted language, we will merely move on to the next stage
- We can develop a Docker image that combines compiled binary or interpreted code with the required runtime and dependencies either manually or through continuous integration orchestration. Our new component artifacts are images

We have completed the building step and the completed image, with all components, which must now be delivered to production. But first, we need to make sure it works and is healthy (Will it work? How about the show?). Using the image artifact we built, we can run these tests in various contexts.

Shipping

Containers make it easier to share generated artifacts. Some of the new steps are as follows:

- Our build host system has the image produced (or even on our laptop). We will save this item to an image registry so that it can be used in future workflow procedures
- Docker Enterprise provides integrations on Docker Trusted Registry during continuous integration stages to follow discrete processes from the initial push, image scanning to hunt for vulnerabilities, and different image pulls from different environments
- Docker Engine manages all pushes and pulls triggered by Docker clients

We need to run containers using environment variables or configurations for each step during integration and performance tests now that the image has been shipped to different environments.

Running

So, we have created new artifacts that are simple to share between environments, but we need to put them into production. Some of the advantages of containers for our applications are as follows:

- Docker Engine will execute our containers (processes) in all environments, but that is it. More than Docker Engine, we do not need any other software to run the image successfully (naturally, we have simplified this idea because we will need volumes and external resources in many cases)
- Suppose our image passed all of the workflow's tests. In that case, it is ready for production, and this step will be as simple as deploying the image created in the previous environment with all of the appropriate arguments and environment variables or configurations
- All of these procedures would have been done safely, with resilience, using internal load balancers, and with appropriate replicas, if our environments had been orchestration-managed using Swarm or Kubernetes, among other features that this type of platform delivers.

Windows Containers

Containers originated on Linux, but we can now run and orchestrate them on Windows. In Windows 2016, Microsoft introduced containers into the operating system. They cemented their cooperation with Docker to produce a container engine that executes containers natively on Windows with this release.

Following a few releases, Microsoft decided to take two approaches to containers on Windows, which are as follows:

- Windows Server Containers (WSC), also known as process containers, are a type of container that runs on Windows Server
- Containers for Hyper-V

Customizing Docker

Docker's behavior can be controlled at both the daemon and client levels. Command-line parameters, environment variables, and configuration file definitions can all be used to run these settings.

Customizing the Docker Deamon

Several configuration files and variables control the behavior of the Docker daemon:

key.json: This file includes the daemon's unique identity; it is the daemon's public key that uses the JSON web key format.

deamon.json: The Docker daemon configuration file is daemon.json. Its parameters are all stored in JSON format. The daemon's flags will be available to adjust its behavior in a key-value (or collection of values) format. Configs in the systemd service file must not conflict with choices set in the JSON file;

otherwise, the daemon will not start.

Environment Variables: HTTPS_PROXY, HTTP_PROXY, and NO_PROXY (or lowercase) are environment variables that control how the docker daemon and the client behind the proxies are used. The following text for HTTPS_PROXY (the same configuration may be applied to HTTP_PROXY) can be found in the docker daemon systemd unit config files, for example, /etc/systemd/system/docker.service.d/http-proxy.conf:

```
Environment=”HTTPS_PROXY=https://proxy.example.com:443/”

“NO_PROXY=localhost,127.0.0.1,docker-registry.example.com,.corp”
```

Docker Client Customization

The client will save its configuration to Docker in the user's home directory. The Docker client looks for its configurations in a configuration file.

($HOME/.docker/config.json on Linux or %USERPROFILE%/.docker/config.json on Windows)

If we need to connect to the internet or other external services, we will set a proxy for our containers in this file.

Docker Security

Docker provides a client-server environment. Few things can be done on the client-side to improve our ability to access the environment.

Filesystem security at the operating system level must be used to secure configuration files and certificates for distinct clusters on hosts. However, as you may have seen, a Docker client requires a server to accomplish anything with containers. The Docker client is merely a means of connecting to servers. Client-server security is essential in this scenario.

Docker Daemon Security

The following principles underpin Docker container runtime security:

- The kernel provides container security
- The runtime itself is an attack surface
- Runtime security based on the operating system

Namespace

We have discussed kernel namespaces and how they implement the requisite container separation. Every container uses the following namespaces:

- pid (Process ID – PID): Process isolation

- net (Networking - NET): Manages network interfaces
- IPC resources (InterProcess Communication - IPC) are managed by ipc
- mnt (Mount – MNT): Manages filesystem mount points
- uts (Unix Timesharing System - UTS) isolates kernel and version IDs

Linux Security Modules

Linux operating systems come with security tools. They are installed and configured by default in certain out-of-the-box installations, while they require further administrator intervention in others.

Docker Content Trust

Docker Content Trust is Docker's mechanism for improving content security. It will give image ownership as well as immutability verification. This option, applied during Docker runtime, will aid in content execution hardening. Only selected images will be able to execute on Docker hosts. There will be two degrees of security as a result of this:

- Only signed photos are permitted
- Allow just selected people or groups/teams to sign photographs

Chapter 03: Docker Community Edition Installation and Configuration

Introduction

Docker is a free and open platform for building, delivering, and operating apps. It allows you to decouple your apps from your infrastructure, allowing you to swiftly release software. You can manage your infrastructure the same way you control your applications with Docker. You may drastically minimize the time between writing code and executing it in production by leveraging Docker's approaches for shipping, testing, and deploying code quickly.

Docker is available for download and installation on a variety of systems. There are two different Docker editions:

- Community Edition
- Enterprise Edition

The Enterprise Edition is not free and comes with a few additional features and benefits. However, the Community Edition is the free and open-source version of the Docker Engine.

It is important to note that in terms of the core functionality, only the ability to run containers on a server and some of the core features of Docker, like Docker Swarm, networking, and security.

Docker Community Edition is as powerful as the Docker Enterprise Edition, and in fact, it gets all of the Docker Engine updates. Therefore, it is not like you will find it with some pieces of software, where the open-source and free version is several versions behind the new version. In terms of the core functionality, they both include the same features and are updated on the same schedule.

Docker

Docker is a technology or platform that makes it easier to create, deploy, package, and ship applications and their components like libraries and other dependencies. Its main goal is to automate the application deployment process and virtualization at the operating system level on Linux. It enables several containers to run on the same hardware, resulting in increased productivity and

the isolation of programs, and the ease of configuration.

Docker is available for download and installation on a variety of systems:

- **Docker Desktop for Mac** – A native application that offers all Docker tools to your Mac, leveraging the macOS sandbox security mechanism
- **Docker Desktop for Windows** – A native Windows application that installs all of the Docker tools on your machine
- **Docker for Linux** - Install Docker on a machine that already runs a Linux distribution

Docker Platform

Docker is a free and open-source platform for building, delivering and executing applications. It allows you to bundle and run an application in a container, a loosely isolated environment. Because of the isolation and security, you can operate multiple containers on the same host simultaneously. Containers are small and include everything needed to operate an application, so you do not have to rely on what is already on the host. While working, you can quickly share containers and ensure that everyone gets the same container that operates the same way.

Docker provides tools and a framework for managing container lifecycles:

- Containers can be used to develop your application and its supporting components
- Your application's container becomes the unit for distributing and testing it
- Deploy your application as a container or an orchestrated service into your production environment when you are ready. Whether your production environment is a local data center, a cloud provider, or a hybrid of the two, this works the same

What Is Docker Used For?

Fast and consistent delivery of your applications

Docker simplifies the development process by allowing developers to work in standardized settings while employing local containers to provide their apps and services. Continuous Integration and Continuous Delivery (CI/CD) workflows benefit greatly from containers.

Consider the following circumstance as an example:

- Your developers write code locally and use Docker containers to share it with their co-workers

- They use Docker to deploy their applications and run automated and manual tests in a test environment
- Developers can repair defects in the development environment before deploying them to the test environment for testing and validation
- When the testing is thorough, it is just a matter of releasing the revised image to the production environment to get the repair to the customer

Responsive deployment and scaling

The container-based Docker platform enables extremely portable workloads. Docker containers can run on a developer's laptop in a data center on real or virtual machines, cloud providers, or hybrid environments.

Because of Docker's mobility and lightweight nature, it is also simple to dynamically manage workloads, scaling up or down apps and services in near real-time as business needs dictate.

Running more workloads on the same hardware

Docker is a lightweight and quick application that offers a practical and cost-effective alternative to hypervisor-based virtual machines, allowing you to use your computational resources better. It is ideal for high-density situations and small and medium deployments where more can be accomplished with fewer resources.

Docker Architecture

Docker is a lightweight and quick application that offers a practical and cost-effective alternative to hypervisor-based virtual machines, allowing you to use your computational resources better. It is ideal for high-density situations and small and medium deployments where more can be accomplished with fewer resources.

Docker Daemon

The Docker daemon (dockerd) handles Docker objects, such as images, containers, networks, and volumes, by listening for Docker API requests. To manage Docker services, a daemon can communicate with other daemons.

Docker Client

Many Docker users interact with Docker, primarily through the Docker client (Docker). When you run commands like docker run, the client transmits them to dockerd that executes them. The docker command uses the Docker API, and the Docker client can communicate with many daemons.

Docker Desktop

Docker Desktop is a simple-to-use program for building and sharing containerized applications and microservices on your Mac or Windows computer. It includes Docker Compose, Docker Content Trust, Kubernetes, Credential Helper, and the Docker daemon (dockerd).

Docker Registries

Docker images are stored in a Docker registry. Docker Hub is a public registry that anybody may use, and Docker is set up by default to look for images. It is also possible to run your own private registry.

The relevant images are pulled from your configured registry using the docker pull or docker run commands. When you run the docker push command, your image gets pushed to your configured registry.

Docker Objects

When you use Docker, you create and use images, containers, networks, volumes, plugins, etc. This section provides a quick overview of a few of those items.

Images

An image is a read-only template with Docker container creation instructions. A lot of the time, an image is based on another image with some tweaking. For example, you could create an image based on the Ubuntu image and include the Apache web server, your application, and the configuration data required to execute your application.

You can either make your own photos or rely on those generated by others and stored in a registry. You create your own image by writing a Dockerfile with a simple syntax for outlining the procedures required to create and operate the image. Each Dockerfile instruction builds a layer in the image, and only the changed layers are rebuilt when you edit the Dockerfile and rebuild the image. Compared to other virtualization technologies, this is part of what makes pictures so light, tiny, and fast.

Containers

A container is an image that can be run. Using the Docker API or CLI, you can create, start, stop, move, or destroy a container. You can attach storage to a container, connect it to one or more networks, or even construct a new image based on its existing state.

By default, a container is often well separated from other containers and its host machine. You have control over how isolated a container's network, storage, and other underlying subsystems are from other containers and the host machine.

Docker Engine Overview

Docker Engine is a containerization tool that you may use to develop and containerize your apps. It is a client-server program that includes:

- dockerd is a long-running daemon process on a server.
- APIs define interfaces via which programs can communicate with and command the Docker daemon.
- A client docker with a command-line interface (CLI).

The CLI employs Docker APIs to control or communicate with the Docker daemon through scripting or direct CLI instructions. Many other Docker applications use the underlying API and CLI. The daemon creates and manages docker objects, such as images, containers, networks, and volumes.

Docker Engine – CentOS (Community)

The CentOS (Community) distribution's Docker Engine is the best option to install the Docker platform on CentOS-based Linux installations. Simplify Docker provisioning and setup to reduce time to value when developing and deploying container-based apps.

Docker Engine can be installed on bare metal or cloud infrastructure and is offered for free (Community) and as a subscription in Docker Enterprise.

Features and Benefits

- Installing and configuring an optimal Docker environment for CentOS on bare metal servers and virtual machines is simple
- The most recent version of the Docker platform includes orchestration (clustering and scheduling), runtime security, container networking, and volumes
- Docker Engine (Community) is offered as a free download with a monthly Edge or quarterly Stable release with community assistance
- Docker Enterprise features quarterly releases one year of maintenance per version, and enterprise-level support with SLAs

Install using convenience script

Docker provides a convenience script at get.docker.com to install Docker into development environments easily and non-interactively. The convenience script

is not recommended for production environments, but it can be used as a starting point for creating a custom provisioning script. See the install using the repository instructions for more information on installing using the package repository. The script's source code is available in the docker-install repository on GitHub, which is open-source.

Before running scripts downloaded from the internet, always inspect them. Before installing, familiarize yourself with the convenience script's potential hazards and limitations:

- To run the script, you will need root or sudo capabilities
- The script tries to detect your Linux distribution and version and configure your package management system for you. However, most installation parameters are not customizable
- Without prompting for confirmation, the script installs requirements and suggestions. Depending on your host machine's existing configuration, this may install many packages.
- The script installs the most recent stable versions of Docker, containerd, and runc by default. When using this script to provision a computer, large Docker version upgrades may occur unexpectedly. Before deploying to your production systems, always test (large) upgrades in a test environment
- The script is not intended to upgrade a Docker installation that already exists. Dependencies may not be updated to the intended version when using the script to update an existing installation, resulting in the use of obsolete versions

Docker Storage Driver

Docker uses a pluggable design to support a variety of storage drivers, which governs the storage and management of images and containers on your Docker host.

Running a Container

A container is a typical operating system process except that it is isolated from the host and has its own file system, networking, and isolated process tree.

The docker run command is used to run an image within a container. This syntax is:

```
docker run [OPTIONS] IMAGE[:TAG] [COMMAND]   [ARG…]
```

IMAGE – The container image to run. By default, these are pulled from Docker Hub.

TAG – A specific image tag, usually used to pull a specific version.

COMMAND – Command to run inside the container.

ARG – Arguments to pass when running the command.

Used Options

Following are some commonly used options to run a container:

-d – It runs the container in dedicated mode. The docker run command will exit immediately, and the container will run in the background.

--name – A container is assigned a random name by default, but you can give it a more descriptive name with this flag.

--restart – It specifies when the container should be automatically restarted.

- **no (default) –** Never restart the container
- **on-failure –** Only if the container fails (exits with a non-zero exit code)
- **always –** Always restart the container whether it succeeds or fails. Also, it starts the container automatically on daemon startup
- **unless-stopped –** Always restart the container whether it succeeds or fails. Also, it starts the container on daemon startup unless the container was manually stopped
- **-p <host port>:<container port> -** Expose a container's port by mapping it to a port on the host. You can use –p multiple times to map multiple ports/
- **--rm –** It automatically removes the container when it exits and cannot be used with –restart
- **--memory –** Hard limit on memory usage
- --memory-reservation – A soft limit on memory usage. The container will be restricted to this limit if Docker detects memory contention (not enough memory on the host)

Logging Drivers

Logging drivers are a pluggable framework for accessing log data from services and containers in Docker, and Docker supports a variety of logging drivers.

Configuring Logging Drivers

Docker has several logging tools built in to enable you to collect data from running containers and services; logging drivers are the name for these mechanisms. Each Docker daemon has a default logging driver used by all containers until you specify a different logging driver, or "log-driver," for short.

Docker's default logging driver is json-file, which internally stores container logs as JSON. You can implement and utilize logging driver plugins and the logging drivers offered with Docker.

Introduction to Docker Swarm

Swarmkit is used to build the Docker Engine's cluster management and orchestration functionality. It is a distinct project that implements Docker's orchestration layer and is integrated into Docker directly.

Multiple Docker hosts run in swarm mode and act as managers (to control membership and delegation) and workers in a swarm (which run swarm services). A Docker host can be a manager, a worker, or both simultaneously. You define the optimal state of a service when you build it (number of replicas, network and storage resources available to it, ports the service exposes to the outside world, and more). Docker strives to keep the desired state; for example, it schedules a worker node's duties on other nodes if that node becomes unavailable. Unlike a solo container, a task is a running container part of a swarm service and managed by a swarm manager.

Nodes

A node is a Docker engine instance that is part of the swarm, which can also be thought of as a Docker node. One or more nodes can run on a single physical computer or cloud server; however, production swarm deployments usually have Docker nodes spread across numerous physical and cloud servers.

Services and Tasks

A service defines the tasks to be executed on the management or worker nodes. It is the swarm system's central structure and the principal point of user contact with the swarm.

You select the container image to use and which commands to run inside running containers when you build a service.

The swarm manager distributes several replica jobs among the nodes in the replicated services model based on the scale you set in the desired state.

The swarm conducts one task for each global service on every accessible node in the cluster.

A task contains a Docker container and the commands to run inside it. It is the swarm's atomic scheduling unit. Manager nodes assign jobs to worker nodes according to the number of replicas established in the service scale. A job that

has been assigned to a node cannot be moved to another node. It can only run on the node that has been assigned to it, or it will fail.

Load Balancing

The swarm manager uses ingress load balancing to expose the services you want to make available outside the swarm. The swarm manager can automatically provide a PublishedPort to the service, or you can manually specify one. Any unused port can be specified, and if you do not specify a port, the swarm manager assigns a port between 30000 and 32767 to the service.

External components, such as cloud load balancers, can access the service via the PublishedPort of any node in the cluster, regardless of whether that node is actively performing the service's duty. Ingress connections are routed to a running task instance by all nodes in the swarm.

Swarm mode contains an internal DNS component that assigns a DNS entry to each service in the swarm. The swarm manager uses internal load balancing to distribute requests among services within the cluster depending on their DNS names.

Configure Swarm Nodes

You can add some worker nodes to the swarm with a manager setup:

- Install Docker CE on both worker nodes
- Get a join command from the manager: Run docker swarm join-token worker on the manager node to get a join command
- Run the join command on both workers: Copy the join command from the manager and run it on both workers
- Verify both workers have successfully joined the swarm: Run docker node ls on the manager and verify that you can see the two worker nodes listed

Swarm Backup and Restore

It is always a good idea to back up critical data in a production environment.

Backing up Docker swarm data is fairly simple. To back up, do the following on a swarm manager:

- Stop the Docker service
- Backup all the data in the directory /var/lib/docker/swarm
- Start the Docker service

To restore a previous backup, follow the given steps:

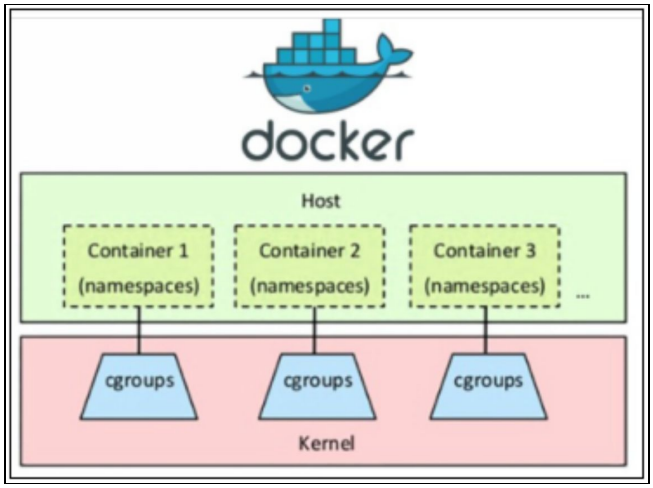
- Stop the Docker service

- Delete any existing files or directories under /var/lib/docker/swarm
- Copy the backed-up files to /var/lib/docker/swarm
- Start the Docker service

Namespace and Cgroups

Cgroups and namespaces are two basic Linux Kernel technologies used to run Linux Containers.

Docker establishes a set of namespaces and control groups for the container when you start it.



Namespace

Namespace is a Linux technology that allows processes to be listed in terms of the resources that they see. It can prevent different processes from interfering or interacting with one another.

Docker uses namespace to isolate containers, allowing containers to operate independently and securely.

Docker uses namespaces such as the following to isolate resources for containers:

- Each user namespace has its own set of user and group IDs for process isolation. This indicates that a process can have root privilege only in its own user namespace and not in any other user namespaces
- pid – A PID namespace assigns a set of PIDs to processes distinct from the PIDs assigned to processes in other namespaces. PID 1 is assigned to the first process created in a new namespace, and subsequent PIDs are assigned to child processes. If a child process has its own PID namespace, it has PID 1 in that namespace as well as PID 1 in the parent process' namespace

- net – A network namespace contains its own network stack, which includes a private routing table, IP addresses, socket listing, connection tracking table, firewall, and other network resources
- ipc – A namespace for inter-process communication (IPC) has its own IPC resources, such as POSIX message queues
- mnt – A mount namespace has its own list of mount points that are visible to the namespace's processes. This means that filesystems in a mount namespace can be mounted and unmounted without affecting the host filesystem
- uts – A UNIX TimeSharing (UTS) namespace allows distinct processes on the same system to appear to have different host and domain names
- user namespace – Requires special configuration and allows container processes to run as root inside the container while mapping that user to an unprivileged user on the host

Cgroups

Control groups are another technology used by Docker Engine on Linux. A cgroup restricts an application's access to a limited set of resources. Docker Engine uses control groups to share available hardware resources among containers and enforce restrictions and constraints. You can, for example, limit the amount of memory available to a specific container.

Chapter 04: Image Creation, Management, and Registry

Introduction

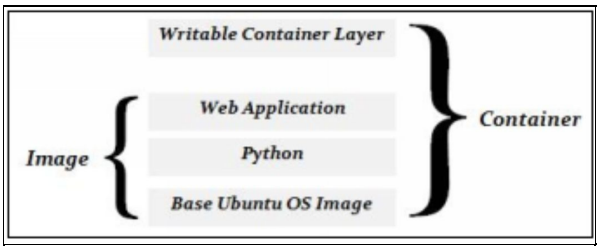
This chapter will learn about the Docker images, management, and registry. In a Docker container, a Docker image is a file that contains the code to be executed. Docker images, like templates, operate as a set of instructions for building a Docker container. The Management feature allows Organization owners to restrict which images their developers can obtain from Docker Hub. Docker images can be kept in a Docker Registry.

Docker Images

A Docker image contains the code and components needed to run software in a container.

Images and Containers employ a tiered file system, in which each layer only contains differences from the preceding layer.

The image consists of more read-only layers, while the container adds one additional writable layer.



The Components of a Dockerfile

Dockerfiles

If you want to create your image, you can do so with a Dockerfile.

A Dockerfile is a collection of instructions for building a Docker image. The terms "directives" and "instructions" are used to characterize these instructions.

FROM:

Sets the foundation image for a new build stage. It is usually the first directive in a Dockerfile (ARG, on the other hand, can be inserted before).

ENV:

Configure the environment variables. These are visible to the container during runtime and referenced in the Dockerfile.

RUN:

Running a command inside the new layer and committing the changes creates a new layer on top of the existing layer.

CMD:

At execution time, specify a default command for running a container.

EXPOSE:

Documents which port(s) are intended to be published when running a container.

WORKDIR:

Sets the current working directory for subsequent directives such as ADD, COPY, CMD, ENTRYPOINT, etc. It can be used multiple times to change directories throughout the Dockerfile. You can also use a relative path, which sets the new working directory relative to the previous working directory.

COPY:

Files from the local machine should be copied to the image.

ADD:

Similar to COPY, but can also pull files using a URL and extract an archive into loose files in the image.

STOPSIGNAL:

Specify the signal that will be used to stop the container.

HEALTHCHECK:

Specify a command to run to perform a custom health check to verify that the container is working properly.

Multi-Stage Builds

Docker supports the ability to perform multi-stage builds. Multi-Stage builds have more than one FROM directive in the Dockerfil, with each FROM directive starting a new stage.

Each stage begins an entirely new set of file system layers, allowing you to selectively copy only the files you need from previous layers.

Use the –-form flag with COPY to copy files from a previous stage.

```
COPY –-from=0 …
```

You can also name your stages with FROM…AS then reference the name with COPY –-from:

```
FROM <image> AS stage1

…

FROM <image> AS stage2

COPY –-from=stage1 …
```

Flattening A Dockers Image To A Single Layer

Sometimes, images with fewer layers can perform better. You may want to take an image with many layers and flatten it into a single layer in a few cases.

Dockers do not provide an official method for doing this, but you can accomplish it by doing the following:

- Run a command from the image
- Export the container to an archive using docker export
- Import the archive as a new image using docker import

The resulting image will have only one layer. You can flatten an image to run a container, then export the container to an archive using docker export, and then import the archive as an image using docker import. So we are running a container off the image and then turning the container into another image that will only have 1 layer. To begin, make sure you are in your home directory, then navigate to / and create an alpine-hell project directory. To create an image, you will need a Dockerfile.

Docker Registries

Docker images are stored and distributed using a Docker registry. We have already pulled images from the default public registry, Docker Hub.

You can also create your registries using Dockers open-source registry software or Docker Trusted Registry, the non-free enterprise solution.

To create a basic registry, run a container using the registry image and publish port 5000.

Registry Server

- Distribution - Ubuntu 18.04 Bionic Beaver LTS
- Size – Small

Securing A Registry

By default, the registry is completely unsecured, does not use TLS, and does not require authentication.

You can take some basic steps to secure your registry:

- Use TLS with a certificate
- Require user authentication

Using Docker Registries

Docker Pull IMAGE[: TAG]

Download an image from a registry to the local system.

Docker login REGISTRY

Authentication with a remote registry.

When working with Docker registries, if the registry is not specified, the default registry will be used (Docker Hub).

There are multiple ways to use a registry with a self-signed certificate.

- Turn off certificate verification (very insecure)
- Provide the public certificate to the Docker engine

To push and pull images from your private registry, tag the images with the registry hostname (and optionally, port).

<registry public hostname>/<image name>:<tag>

Docker Push IMAGE

Upload the image to a remote registry.

Chapter 05: Orchestration

Introduction

Docker orchestration is a set of strategies and tools for the large-scale management of Docker containers. Container orchestration features, such as provisioning containers, scaling up and down, managing networking, load balancing, and other problems, become necessary as containerized applications scale to multiple containers.

There are two primary options for Docker orchestration at the moment:

Kubernetes

Kubernetes is the de-facto container orchestration standard, and it is widely used to orchestrate Docker containers. Kubernetes is a powerful yet complicated platform that requires a long learning curve. Other orchestrators can be used for Docker orchestration, such as Nomad and OpenShift Container Platform (which includes Kubernetes at its heart but enhances its capabilities).

Swarm and Other Docker-native Tools

The Docker ecosystem provides several orchestration tools that are less powerful than Kubernetes but are simple to use and available to Docker users. Docker Swarm, a lightweight container orchestrator, and Docker Compose, which executes multi-container applications automatically, are two examples.

Docker Swarm

Swarm mode in Docker allows you to manage a cluster of Docker Engines directly from the Docker platform. Create a swarm, deploy application services to a swarm, and govern swarm behavior with the Docker CLI.

Docker will soon support both Kubernetes Guide and Docker Swarm, allowing Docker users to orchestrate their container workloads using either Kubernetes or Swarm.

Swarm can assist developers and IT administrators in the following ways:

- Assign tasks to groups of containers and coordinate between containers
- Check the health of individual containers, and their lifecycles should be managed
- If one or more nodes fail, provide redundancy and failover

- Depending on the load, adjust the number of containers
- Roll out software updates to several containers at the same time

Docker Swarm Concepts

Swarmkit

Swarmkit is a distinct project that provides Dockers' orchestration layer and is used to create Docker swarm mode directly within Docker.

Swarm

A swarm comprises numerous Docker hosts that work together as managers and workers in a swarm mode.

Task

The swarm manager distributes several tasks among the nodes based on your chosen service size. A task contains a Docker container and the commands to run inside it. A job that has been assigned to a node cannot be moved to another node. It can only run on the node that has been assigned to it, or it will fail.

Service

A service is a definition of the tasks that will be performed on the management or worker nodes. You select the container image to use and which commands to run inside running containers when you build a service. One significant distinction between services and standalone containers is that you can change the configuration of a service, including the networks and volumes to which it is attached, without having to restart it.

Nodes

A swarm node is a single Docker Engine that participates in the swarm. Although you can operate one or more nodes on a single physical computer or cloud server, most production swarm deployments use Docker nodes spread over numerous computers.

Manager Nodes

Assign tasks to worker nodes in the form of units of work. The manager nodes also perform the orchestration and cluster management functions.

Leader Node

Management nodes use the Raft consensus process to elect a single leader to undertake orchestration activities.

Worker Nodes

Worker nodes are responsible for receiving and completing tasks assigned by manager nodes. Manager nodes run services as worker nodes by default. Each

worker node has an agent that operates and reports to its manager node on its tasks.

Load Balancing

The swarm manager employs ingress load balancing to expose the Docker swarm's services to the outside world. The swarm controller gives the service a customizable PublishedPort. External components, such as cloud load balancers, can access the service using the PublishedPort of any node in the cluster, regardless of whether that node is actively performing the services' duty. Ingress connections are routed to a running task instance by all nodes in the swarm. The swarm manager uses internal load balancing to distribute requests among services within the cluster depending on their DNS names.

Features

Cluster Administration with Docker Engine: Create a swarm of Docker Engines where you can deploy application services using the Docker Engine CLI. You do not require any additional orchestration software to establish or manage a swarm.

Decentralized Design: The Docker Engine handles any specialization at runtime rather than distinguishing between node responsibilities at deployment time. The Docker Engine can be used to deploy both manager and worker nodes. This means that a single disk image can be used to create a whole swarm.

Declarative Service Model: Docker Engine takes a declarative approach to defining the desired state of the application stack's multiple services. You could, for example, describe an application that combines a web front-end service with message queueing services and a database backend.

Scaling: You can specify how many jobs you want to run for each service. When you increase or decrease the size of your swarm, the swarm manager automatically adjusts by adding or removing tasks to keep the appropriate state.

Desired state reconciliation: The swarm manager node regularly monitors the cluster state and reconciles any inconsistencies between the actual state and the desired state you specified. If you set up a service to run ten replicas of a container and a worker machine hosting two of those replicas crashes, the manager produces two new replicas to replace the ones that crashed. The swarm manager distributes the fresh clones to active workers.

Multi-host Networking: For your services, you can create an overlay network. When the swarm manager initializes or updates the application, it automatically

allocates addresses to the containers on the overlay network.

Service Discovery: Swarm manager nodes assign a unique DNS name to each service in the swarm and load balance running containers. Through a DNS server embedded in the swarm, you can query every container running in the swarm.

Load Balancing: You can expose service ports to an external load balancer. Internally, the swarm allows you to determine how service containers are distributed amongst nodes.

Secure by Default: To secure connections between itself and other nodes, each node in the swarm uses TLS mutual authentication and encryption. Self-signed root certificates or certificates from a bespoke root CA are both options.

Rolling Updates: You can apply service upgrades to nodes in stages throughout the rollout. The swarm manager allows you to control the time it takes for services to be deployed to different groups of nodes. You can revert to a prior version of the service if something goes wrong.

Locking and Unlocking a Swarm Cluster

Docker can securely store encryption keys. These encryption keys are used to encrypt important cluster data; however, they are stored on swarm manager disks unencrypted by default. Autolock improves the security of these keys, but it necessitates that each manager is unlocked every time Docker restarts.

Autolock

Docker swarm encrypts sensitives data for security reasons, such as:

- Raft logs on swarm managers
- TLS communication between swarm nodes

By default, Docker automatically manages the keys used for this encryption, but they are stored unencrypted on the managers' disks.

Autolock is a feature that automatically locks the swarm, allowing you to manage the encryption keys yourself. This gives you control of the keys and allows more security.

Enable and Disable Autolock

When you initialize a new swarm with the --autolock flag, you can enable auto-lock. The following commands are used to enable and disable auto-lock on a running swarm, respectively:

- **docker swarm update --autolook=true** //enable
- **docker swarm update --autolook=false** //disable

High Availability in a Swarm Cluster

It is better to have multiple swarm managers to build a high-availability and fault-tolerant swarm cluster.

Manager nodes are critical components for controlling and storing the swarm state when running a swarm of Docker Engines. To correctly install and maintain the swarm, it is crucial to understand several key properties of manager nodes.

Operate Manager Node in a Swarm

Swarm manager nodes use the Raft Consensus Algorithm to govern the swarm state. To manage a swarm, you only need to understand a few basic Raft ideas.

The number of management nodes is unrestricted. It is a trade-off between performance and fault-tolerance when deciding how many manager nodes to use. When you add manager nodes to a swarm, it becomes more fault-tolerant. On the other hand, additional management nodes slow down write speed since more nodes must accept proposals to change the swarm state. As a result, there will be greater network round-trip traffic.

Raft requires a majority of managers, commonly known as the quorum, to agree on proposed swarm updates such as node additions and deletions. The same limits apply to membership operations as they do to state replication.

Quorum

A Quorum is the majority (more than half) of the manager in a swarm. For example, for a swarm with 5 managers, the quorum is 3.

A quorum must be maintained to make changes to the cluster state. If a quorum is unavailable, nodes cannot be added or removed, new tasks cannot be added, and existing tasks cannot be changed or moved.

Introduction to Docker Services

A service is used to run an application in Docker Swarm. A service specifies a set of one or more replica tasks. These tasks will be distributed automatically across the cluster nodes and executed as containers.

Deploy Services to a Swarm

Swarm services employ a declarative approach, which means you declare the service's desired state and trust Docker to keep it there. The state includes (but is not limited to) the following information:

- The service containers should run under the image name and tag
- How many containers will be included in the service
- Whether any ports are open to clients who are not part of the swarm
- Whether or not the service should be started automatically when Docker is launched
- Time the service is resumed, the specific behavior that occurs (such as whether a rolling restart is used)
- Characteristics of the nodes that can run the service (such as resource constraints and placement preferences)

How do Services Work

When Docker Engine is in swarm mode, you must construct a service to deploy an application image. Service is frequently used to represent a microservice within a bigger application. An HTTP server, a database, or any other executable software that you want to operate in a distributed environment are examples of services.

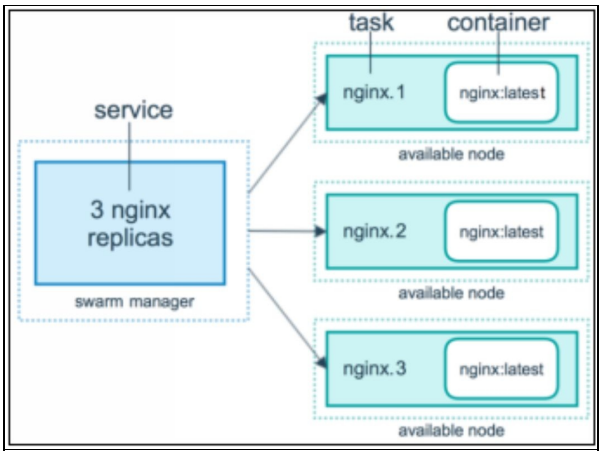
You select the container image to use and which commands to run inside running containers when you build a service. You can additionally provide service choices, such as:

- The location where the swarm makes its service available to others who are not part of the swarm
- An overlay network allows the service to communicate with other swarm services
- Reservations and CPU and memory limits
- A policy of continuous updating
- The number of image replicas that will run in the swarm

Services, Tasks, and Containers

The swarm manager accepts your service definition as the expected state for the service when you deploy it to the swarm. The service is then scheduled as one or more replica tasks on nodes in the swarm. On the swarms' nodes, the jobs run independently of one another.

Consider the following scenario: you wish to load balance three HTTP listener instances. An HTTP listener service with three replicas is depicted in the diagram below. The listener has three instances, each of which is a swarm task.



Task and Scheduling

Within a swarm, a job is the atomic unit of scheduling. The orchestrator realizes the desired service state by scheduling tasks when you declare a desired service state by creating or modifying the service. For example, you might create a service that tells the orchestrator to keep three HTTP listener instances running at all times. As a result, the orchestrator creates three tasks. The scheduler fills each job slot by launching a container. The job is instantiated in the container. The orchestrator launches a new replica task that spawns a new container if an HTTP listener task fails its health check or crashes.

A task is a system that only works in one direction. It goes through a sequence of states in a predictable order: assigned, prepared, running, etc. If the job fails, the orchestrator deletes the task and its container and replaces it with a new task that conforms to the services' desired state.

Dockers' swarm mode is based on a general-purpose scheduler and orchestrator. The containers that the service and task abstractions implement are completely unknown to them. Other sorts of tasks, such as virtual machine tasks or non-containerized process tasks, might theoretically be implemented. The scheduler and orchestrator are not concerned with the tasks' kind. However, Docker only supports container tasks in its present version.

Templates

Templates can be used to give somewhat dynamic values to some flags with **docker service create.**

The following flags accept templates:

- --hostname
- --mount
- --env

Replicated vs. Global Services

Replicated and global service deployments are the two forms of service deployments.

You provide the number of identical jobs you want to run for a replicated service. For instance, suppose you want to set up an HTTP service with three copies offering the same information.

Global service performs the same task on all nodes. There is no set number of jobs to complete. When you add a new node to the swarm, the orchestrator creates a job, which is then assigned to the new node via the scheduler. Monitoring agents, anti-virus scanners, and other types of containers that you wish to run on every node in the swarm are good candidates for global services.

Scaling Service

You can scale one or more replicated services up or down to the desired number of replicas using the scale command. This command will not work on services that are in global mode. The command will be executed quickly; however, scaling the services may take some time. Set the scale to 0 to halt all service clones while keeping the service running in the swarm.

Docker Compose

Docker Compose allows you to orchestrate many containers that interact with one another. A service that handles requests and has a front-end website is an example, as is a service that employs a supporting function like a Redis cache. You can use Docker Compose to split the app code into numerous independently running services that communicate through web requests if you are using the microservices architecture for app development.

Compose is a Docker application that defines and operates multi-container Docker applications. You configure your application's services using Compose and a YAML file. Then you build and start all of the services from your setup with a single command.

Compose is compatible with all environments, including production, staging, development, testing, and continuous integration workflows. In Common Use Cases, you may learn more about each situation.

Docker Stack

Docker Stack is a layer above Docker containers that allows you to orchestrate many containers across various servers. Docker Stack runs on a Docker Swarm, effectively a collection of Docker daemon-running machines grouped to share

resources.

Multiple services, which are containers dispersed over a swarm, can be deployed and logically grouped using stacks. A Stacks' services can be configured to operate many replicas, or clones, of the underlying container. This number will be kept for the duration of the service. It is simple to add or remove clones of a service. Docker Services is great for executing stateless processing applications because of this.

Chapter 06: Storage and Volume

Introduction

When administrators first start working with Docker containers, one of the first things they notice is that containers use non-persistent storage by default. When a container is removed, the container's storage is also deleted. Containerized apps, of course, would be useless if there was no mechanism to enable permanent Docker container storage. Fortunately, persistent storage may be implemented in a containerized system. Although the native storage of a container is non-persistent, it can be connected to storage that is external to the container. Because the external storage is not destroyed when a container is halted, this enables the storing of persistent data.

The first step in building persistent storage for your containers is to figure out what kind of storage system you will be using. Three basic storage options are commonly available in this regard:

- File system storage
- Block storage
- Object storage

Storage Models

Persistent data can be managed using several storage models.

File System Storage

Data is stored as files in a file system, which has been around for decades. Each file is identified by its filename, usually accompanied by attributes. NFS and NTFS are two of the most widely utilized file systems. When it comes to establishing a container to store data durably, file system storage is one of the most straightforward possibilities. Host-based persistence is probably the most well-known example of file system storage (as it relates to containers). The concept of host-based persistence is rather straightforward. A host server is where containers are stored. This host server is self-contained with its own operating system and file system. Containers can be set up to store persistent data in a dedicated folder on the file storage of the host server.

To integrate the container layers into a cohesive file structure, Docker containers often employ a union file system. For data that has to be saved persistently, host-based persistence bypasses the union file system and stores it in the same file

system that is used on the host. The main issue with basic host persistence is that it makes container portability impossible. A dependent resource (persistent storage) is stored on the host server's native file system outside of the container when host persistence is employed. Other types of host persistence have been invented to get around this problem. Multi-host persistence, for example, replicates persistent storage over many host servers using a distributed file system.

Block Storage

Another alternative for container storage is block storage. File system storage, as previously said, arranges data into a hierarchy of files and folders. Block storage, on the other hand, stores data pieces in blocks.

Only the address of a block can be used to identify it, as a block does not have a filename or other metadata of its own. They become meaningful only when blocks are merged with other blocks to make a complete piece of data.

Because of its speed, block storage is often employed in database applications. Block storage is also commonly used for snapshotting, which allows a volume to be rolled back to a precise moment in time without the need to restore a backup. When it comes to containers, block storage is sometimes used as container-defined storage.

Container-defined storage is similar to software-defined storage, except it is designed to work in a containerized environment. Typically, this storage is implemented within a separate storage container.

Object Storage

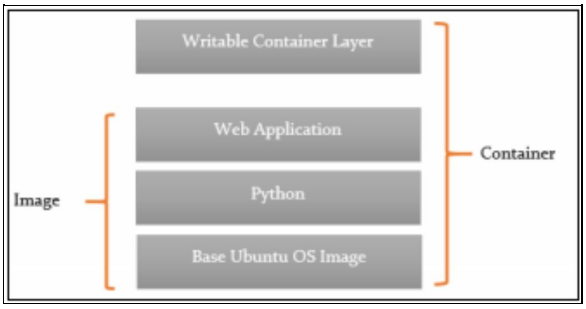
Object storage differs from the file system or block storage in how it works. Data is stored as an object and referenced by an object ID rather than a block address or a file name. Object storage has the advantage of being immensely scalable, allowing for a great deal of freedom when it comes to associating properties with objects. Using object storage has the disadvantage of not performing as well as block storage. Object storage is a common choice for public cloud providers because it is built primarily for scalability. Docker containers can be linked to Amazon Web Services or Microsoft Azure object storage, but this requires the containerized application specifically designed to take advantage of object storage.

Unlike traditional applications, which use a file system or SCSI calls to access data, object storage uses HTTP-based REST calls like Get and Put. As a result,

object storage should be reserved for applications that require massively scaled storage or storage that must span geographical boundaries.

Storage Layers

Docker storage consists of layers, and with the layer file system, the containers that you run consist of multiple layers. You have the image that will consist of multiple layers, and then the container adds an additional writable layer on top of that. Each one of these layers contains only the differences from the previous layer. Therefore, the writable container layer only contains the thing you write to the disk that is changed from the top layer image.



Configuring DeviceMapper

DeviceMapper is a kernel-based framework that powers several of Linux's advanced volume management capabilities. Docker's devicemapper storage driver uses this framework's lightweight provisioning and snapshotting capabilities for image and container management.

Devicemapper support is included in the Linux kernel for systems that support it. However, using it with Docker necessitates some configuration.

Choose the Correct Mount Type

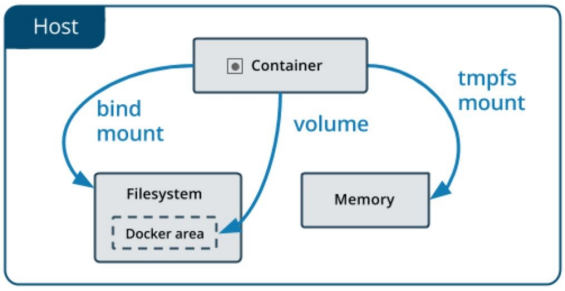
All files created within a container are stored on a writable container layer by default.

- When that container is no longer present, the data is lost. Getting the data out of the container if another process requires it can be challenging
- The writable layer of a container is inextricably linked to the host machine on which it runs. You would not be able to quickly relocate the data to another location
- A storage driver is required to handle the filesystem when writing into the writable layer of a container. Using the Linux kernel, the storage driver provides a union filesystem. When compared to data volumes, which write directly to the host filesystem, this extra layer slows down the performance

Volumes and bind mounts are two alternatives for containers to store files on the host machine, such that they are persistent even after the container is stopped. You can use a tmpfs mount if you are running Docker on Linux. You can also use a named pipe if you are running Docker on Windows.

From within the container, the data appears the same, regardless of which mount you select. The container's filesystem can be accessed as a directory or as a single file.

Consider where the data is stored on the Docker host to see the differences between volumes, bind mounts, and tmpfs mounts.



- Volumes are kept in a Docker-managed section of the host filesystem (/var/lib/docker/volumes/ on Linux). Non-Docker processes should not modify this section of the filesystem. In Docker, volumes are the most efficient way to store data.
- Bind mounts can be kept on the host system in any location. It is possible that they are crucial system files or directories. They can be modified at any moment by non-Docker processes on the Docker host or a Docker container.
- The tmpfs mounts are only held in memory on the host system and are never written to the filesystem.

Volume Mount Type

Docker creates and manages volume mount types. You can create a volume manually with the docker volume create command, or Docker can do it for you when you build a container or service.

When you build a volume on the Docker host, it is saved in a directory. This directory gets mounted into the container when you mount the volume into a container. This works similarly to bind mounts, except that volumes are controlled by Docker and are segregated from the host machine's fundamental functionality.

A volume can be mounted in several containers at the same time. When running

containers are not using a volume, they remain available to Docker and are not instantly destroyed. Docker volume prune is used to remove unused volumes.

A volume can be named or anonymous when it is mounted. When anonymous volumes are mounted inside a container for the first time, Docker assigns them a random name that is guaranteed to be unique within a given Docker host. Apart from the name, named and anonymous volumes act similarly.

Volumes also permit the use of volume drivers, which, among other things, allow you to store your data on remote servers or cloud providers.

Use Cases for Volumes

In Docker containers and services, volumes are the preferred method of storing data. Volumes can be used in a variety of ways, including:

- Data is shared among many containers that are currently executing. A volume is created the first time it is mounted into a container if you do not specifically create it. The volume remains even if the container is stopped or removed. Multiple containers can mount the same read-write or read-only volume at the same time. Only when you remove volumes expressly are they gone
- When there is no certainty that the Docker host will have a specific directory or file structure, volumes allow you to separate the Docker host's configuration from the container execution
- When you would rather store your container's data on a distant host or in the cloud than locally
- Volumes are preferable for backing up, restoring, or migrating data from one Docker host to another. Stop any containers that are utilizing the volume and then back up the volume's directory (for example, /var/lib/docker/volumes/volume-name>)
- When your Docker Desktop application requires high-performance I/O. Because volumes are kept in the Linux VM rather than the host, reads and writes have significantly reduced latency and performance
- When your Docker Desktop application wants native file system functionality. To ensure transaction longevity, a database engine, for example, requires precise control over disk flushing. Bind mounts are remoted to macOS or Windows, where the file systems operate slightly differently, whereas volumes are stored in the Linux VM and can provide these promises

Bind Mount

The bind mount has been offered from Docker's early days. In comparison to volumes, bind mounts offer restricted capabilities. A file or directory on the host machine is mounted into a container when you use a bind mount. The file or directory is referred to by its complete path on the host machine. It is not necessary for the file or directory to already exist on the Docker host. If it does not already exist, it is created on-demand. Bind mounts are fast, but they rely on the filesystem on the host machine having a certain directory layout. Instead of using named volumes, consider employing them while constructing new Docker applications. Bind mounts cannot be managed directly using Docker CLI commands.

tmpfs Mount

On the Docker host or within a container, a tmpfs mount is not persisted on the disk. It can be used by a container to store non-persistent state or sensitive information during its lifespan. Internally, swarm services use tmpfs mounts to mount secrets into service containers.

npipe Mount

For communication between the Docker host and a container, a npipe mount can be utilized. A common use case is to run a third-party tool within a container and use a named pipe to connect to the Docker Engine API.

Storage in a Cluster

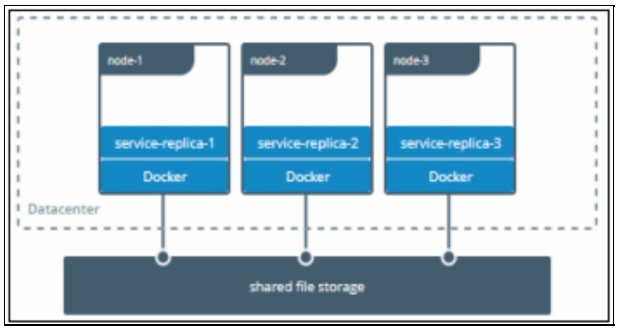
When working with multiple Docker machines, such as a swarm cluster, you may need to share Docker volume storage between those machines.

Some option includes:

- Use application logic to store data in external object storage.
- Use a volume driver to create a volume that is external to any specific machine in your cluster.

Share data among machines

You may need to configure many replicas of the same service to have access to the same files while developing fault-tolerant applications.



When it comes to developing applications, there are various options. One option is to include logic in your application that allows you to save data to a cloud object storage system such as Amazon S3. Another option is to use a driver that allows you to write files to an external storage system, such as NFS or Amazon S3.

Volume drivers separate the application logic from the underlying storage system. For example, suppose your services utilize an NFS driver. In that case, you can upgrade them to use a different driver, such as to store data in the cloud, without modifying the application logic.

Docker Swarm

A Docker Swarm is a collection of physical or virtual machines that have been configured to join together in a cluster and run the Docker application. You can still run the Docker command once a set of machines has been clustered together, but the machines in your cluster will handle them. A swarm manager oversees the cluster's operations, while the machines that have joined the cluster are referred to as nodes.

Features

Cluster administration with Docker Engine: Create a swarm of Docker Engines where you can deploy application services using the Docker Engine CLI. You do not require any additional orchestration software to establish or manage a swarm.

Decentralized design: The Docker Engine handles any specialization at runtime, rather than handling distinction between node responsibilities at deployment time. The Docker Engine can be used to deploy both manager and worker nodes. This means that a single disk image can be used to create a whole swarm.

Declarative service model: Docker Engine takes a declarative approach to defining the desired state of your application stack's multiple services. You

could, for example, describe an application that combines a web front-end service with message queueing services and a database backend.

Scaling: You can specify how many jobs you want to run for each service. When you increase or decrease the size of your swarm, the swarm manager automatically adjusts by adding or removing tasks to keep the appropriate state.

Desired state reconciliation: The swarm manager node regularly monitors the cluster state and reconciles any inconsistencies between the actual state and the desired state you specified. If you set up a service to run ten replicas of a container and a worker machine hosting two of those replicas crashes, the manager produces two new replicas to replace the ones that crashed. The swarm manager distributes the fresh clones to active workers.

Multi-host networking: For your services, you can create an overlay network. When the swarm manager initializes or updates the application, it automatically allocates addresses to the containers on the overlay network.

Service discovery: Swarm manager nodes assign a unique DNS name to each service in the swarm and load balance running containers. A DNS server is embedded in the swarm; you can query every container running in the swarm.

Load balancing: You can expose service ports to an external load balancer. Internally, the swarm allows you to determine how service containers are distributed amongst nodes.

Secure by default: To secure connections between itself and other nodes, each node in the swarm uses TLS mutual authentication and encryption. Self-signed root certificates or certificates from a bespoke root CA are both options.

Rolling updates: You can apply service upgrades to nodes in stages throughout the rollout. The swarm manager allows you to control the time it takes for services to be deployed to different groups of nodes. You can revert to a prior version of the service if something goes wrong.

Chapter 07: Networking

Introduction

One of the advantages of Docker containers and services is that they can be connected to each other or to non-Docker workloads. Docker containers and services do not have to be aware that they are running on Docker or whether or not their peers are Docker workloads. You can use Docker to manage your Docker hosts regardless of whether they run Linux, Windows, or a combination of the two.

This chapter introduces some fundamental Docker networking ideas and prepares you to create and deploy apps that fully utilize these features.

Docker Networking

Docker uses an architecture called the Container Networking Model (CNM) to manage networking for Docker containers.

The CNM utilizes the following concepts:

Sandbox

A Sandbox is an isolated unit containing all networking components associated with a single container. It is usually a Linux Network namespace.

Endpoint

Endpoint connects a sandbox to a network. Each sandbox/container can have any number of endpoints but has exactly one endpoint for each network it is connected to.

Network

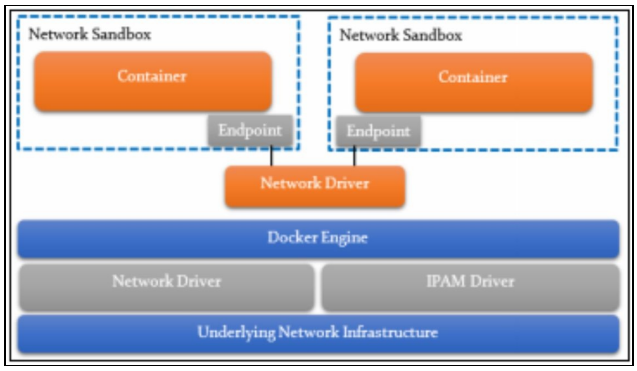
A collection of endpoints connected to one another.

Network Driver

It handles the actual implementation of the CNM concepts.

IPAM Driver

IPAM stands for IP Address Management. It automatically allocates subnets and IP addresses for networks and endpoints.



The networking subsystem in Docker is pluggable and uses drivers. Several drivers come installed by default and enable basic networking functionality:

- **Bridge -** The default network driver is a bridge. This type of network you create if you do not provide a driver. Bridge networks are typically employed when your applications operate in isolated containers and need to connect
- **Host –** It removes network separation between the container and the Docker host for solitary containers and uses the hosts' networking directly
- **Overlay –** The overlay networks are used to connect many Docker daemons and allow swarm services to communicate with one another. Overlay networks can also be used to communicate between a swarm service and a single container or between two single containers running on distinct Docker daemons. Between these containers, our method eliminates the requirement for OS-level routing. They are called Overlay networks
- **ipvlan –** These networks provide users with complete control over IPv4 and IPv6 addressing. For users interested in underlay network integration, the VLAN driver extends on that by offering operators complete control over layer 2 VLAN tagging and even ipvlan L3 routing
- **macvlan –** The macvlan networks allow you to give a container a MAC address, making it look like a physical device on your network. The Docker daemon uses the MAC addresses of containers to redirect communication to them. When dealing with legacy apps that expect to be directly linked to the physical network rather than routed through the Docker hosts' network stack, using the macvlan driver is sometimes the best option
- **None -** Turn off all networking for this container. It is usually combined with a custom network driver.
- **Network plugins -** Docker allows you to install and use third-party network plugins. These plugins can be found on Docker Hub or through

third-party providers

Network Drivers

Docker includes several built-in network drivers, known as native network drivers. These network drivers implement the concepts described in the Container Networking Model (CNM).

The Native Network Drivers are:

- Host
- Bridge
- Overlay
- MACVLAN
- None

Host Network Driver

The Host Network Driver allows containers to use the hosts' network stack directly.

- The container uses the hosts' networking resources directly
- There are no sandboxes; all containers on the host using the host driver share the same network namespace
- No two containers can use the same port(s)
- Use cases – Simple and easy setup, one of only a few containers on a single host

Bridge Network Driver

The bridge network driver uses Linux bridge networks to connect containers on the same host.

This network driver is the default driver for containers running on a single host (i.e., not in a swarm). It creates a Linux bridge of bridge interface for each Docker network to provide connectivity between containers on the same host.

This driver creates a default Linux bridge network called **bridge0.** Containers automatically connect to this bridge into other networks as specified.

The most common use case is the isolated networking among containers on a single host.

Overlay Network Driver

The overlay network driver provides connectivity between containers across multiple Docker hosts, i.e., with Docker swarm.

The network uses a VXLAN data plane, which allows the underlying network

infrastructure (underlay) to route data between hosts in a way that is transparent to the containers themselves.

The overlay driver also automatically configures network interfaces, bridges, etc., on each host as required to support the infrastructure.

The most common use case is that you can use an overlay network if you need the network between containers in a swarm.

MACVLAN Network Driver

The MACVLAN network driver offers a lightweight implementation by connecting container interfaces directly to host interfaces. Instead of using bridge interfaces, it connects container interfaces directly to the host interfaces.

The main drawback is that it is harder to configure and has a greater dependency between MACVLAN and the external network. However, it is more lightweight, and there is less latency.

The use case is if there is a need for extremely low latency or if you need containers that have IP addresses specifically on an external subnet.

None Network Driver

The none network driver does not provide any networking implementation. However, it does provide container sandboxing. Therefore, each container is completely isolated and sandboxed from other containers on the host and the host networking resources themselves.

This driver does not implement any mechanism to allow connectivity between those containers. If you want network connectivity with the none network driver, you have to set everything yourself manually. Therefore, if you want to have full control over your network setup and do everything by yourself, then the non-network driver would allow you to implement that.

The use cases for this driver are simple; when there is no need for container networking or if you want to set up everything manually yourself.

Network Troubleshooting

The section will define some of the techniques and tools that you can use to network troubleshoot networking issues with your Docker setup.

Some of these techniques can also be used for general troubleshooting. When you are using troubleshooting issues, it is very important to gather information about what is going on in your system, and logs can help you do that.

Container Networking

From within the container, the type of network a container uses, whether it is a bridge, an overlay, a macvlan network, or a custom network plugin, is transparent. A network interface with an IP address, a gateway, a routing table, DNS services, and other networking data are available to the container (assuming the container is not using any network driver).

Published Ports

By default, when you use Docker create or docker run to construct or execute a container, none of its ports are exposed to the outside world. Use the --publish or -p flag to make a port available to services outside of Docker or Docker containers that are not linked to the containers' network. This generates a firewall rule that translates a container port to an external port on the Docker host.

IP Address and Hostname

Every Docker network the container connects to is assigned an IP address by default. Since each containers' IP address is assigned from the networks' pool, the Docker daemon effectively serves as a DHCP server. A default subnet mask and gateway are also assigned to each network.

Using --network, the container can only be linked to one network when it starts. You can connect a running container to different networks using docker network connect. When you start a container using the --network flag, you can use the --ip or --ip6 flags to specify the IP address assigned to the container on that network.

When you use docker network connect to connect an existing container to a different network, you can specify the container's IP address on the new network by using the --ip or --ip6 arguments.

Similarly, in Docker, the hostname of a container defaults to the containers' ID. With --hostname, you can override the hostname. You can use the --alias flag to define an additional network alias for the container on an existing network when connecting to it with docker network connect.

DNS Service

A container inherits the hosts' DNS settings from the /etc/resolv.conf configuration file by default. Containers on the default bridge network receive a copy of this file, whereas containers on a custom network use Dockers' embedded DNS server, which routes external DNS requests to the hosts' DNS servers.

Custom hosts in /etc/hosts are not passed down. Add entries to the container hosts file in the docker run reference docs to feed additional hosts into your container. On a per-container basis, you can override these settings.

Chapter 08: Security

Introduction

A security researcher once asked, "Can the images be trusted?". Validating that your images are correct and came from the correct source is one of the key security concerns that must be solved in a container-based system (or maliciously manipulated). One of our security predictions for 2020 was that malicious container images might cause problems for the enterprise pipeline if trusted. You have seen attacks where container images were used to carry out malicious activities like searching for vulnerable hosts and mining cryptocurrencies.

Docker has a feature called "Content Trust" that allows users to confidently deploy images to a cluster or swarm and verify they are the images they claim to be. The Docker Content Trust (DCT) does not monitor your images for modifications throughout the swarm. The Docker client, not the server, performs this one-time check.

Docker Content Trust

Docker Content Trust is a really simple concept at its core. Docker client logic can verify images you retrieve or deploy from a registry server, as long as they are signed on a Docker Notary server of your choice.

Users may validate the integrity of the content they pull using the Docker Notary tool, allowing publishers to sign their collections digitally. In contrast, users can check the integrity of the content they pull. Notary users can give confidence over arbitrary data sets and handle the procedures required to keep material current using The Update Framework (TUF).

Enabling DCT

DCT is turned off by default, and to set it up so that we can sign the pictures we wish to deploy, you will need to perform the following:

- Organize your registry
- Create a Notary server
- Upload a photo to our registry server
- Sign the image that was sent to you
- Set the right environment variables on our management host to allow

Docker commands to verify image signatures

Automate Trust Validation in the CI/CD Pipeline

Verifying whether or not an image is signed (rather than checking for a specific signature) is unlikely to meet in-house security requirements. However, inspecting signatures for any image in your repository allows you to incorporate checks into your CI/CD process. Your team could develop programming to ensure that specific photos were signed by their rightful owners and that only those owners had access to the private keys.

This is a useful technique to ensure that the relevant responsible party has signed off every image sent to production. This makes it more difficult for an attacker to insert a malicious picture into your swarm, whether by social engineering or some other technical means.

Docker Security

When talking about Docker security, there are four important factors to consider:

- The kernel's inherent security, including support for namespaces and cgroups
- The Docker daemon's attack surface
- There are flaws in the container configuration profile, either by default or when users alter it
- The kernel's "hardening" security mechanisms and how they work with containers

Kernel namespace

Docker containers are extremely similar to LXC containers in terms of appearance and security. Docker builds a set of namespaces and control groups for the container behind the scenes when you use docker run to launch a container.

Thanks to namespaces, processes running within a container cannot view, much less affect, processes running in another container or on the host system.

Control Groups

Another important component of Linux Containers is Control Groups, which use resource accounting and limitation techniques. They not only give many helpful metrics, but they also ensure that each container gets its fair amount of memory, CPU, and disk I/O. More significantly, they ensure that a single container cannot bring the system down by depleting one of those resources.

While they do not prohibit one container from accessing or impacting the data

and activities of another, they are necessary to protect against some denial-of-service attacks. They are especially crucial for multi-tenant systems, such as public and private PaaS, to ensure consistent uptime (and performance) even when some apps start acting up.

Docker Daemon Attack Surface

Docker requires the Docker daemon to be running in order to run containers (and apps). Unless you choose Rootless mode, this daemon requires **root** capabilities, so you should be aware of several essential features.

Linux Kernel Capabilities

Docker starts containers with a limited set of capabilities by default. What exactly does that imply?

The binary "root/non-root" dichotomy is transformed into a fine-grained access control scheme via capabilities. Processes (such as web servers) that only need to bind to a port below 1024 do not need to execute as root; instead, the net_bind_service capability can be assigned to them. There is a slew of extra features for practically every situation where root access is required.

Let us have a look at why this is significant for container security.

The SSH daemon, cron daemon, logging daemons, kernel modules, network configuration tools, and other processes run as root on most systems. A container is different because the infrastructure surrounding the container handles practically all of those tasks:

- A single server running on the Docker host is usually in charge of SSH access
- Instead of being a platform-wide facility, cron should be executed as a user process, specialized and optimized for the program that requires its scheduling service
- Typically, log handling is delegated to Docker or third-party services, such as Loggly or Splunk
- Since hardware management is useless, you never need to run udevd or analogous daemons within containers
- Network administration happens outside of the containers as much as possible, ensuring separation of concerns. This means that a container should never need to run ifconfig, route, or ip commands (except when a container is specifically engineered to behave like a router or firewall, of course)

This means that, in the vast majority of circumstances, containers do not require "actual" root rights. As a result, containers can run with a restricted capability set, which means that a container's "root" has far fewer privileges than the real "root." It is possible, for example, to:

- All "mount" operations must be denied
- Access to raw sockets should be denied (to prevent packet spoofing)
- Deny access to filesystem actions, such as establishing new device nodes, changing file owners, or changing characteristics (including the immutable flag)
- Refuse to load the module
- Numerous others

This means that even if an attacker gets to escalate to root within a container, doing major harm or escalating to the host is far more difficult.

It does not affect conventional online programs but significantly decreases the attack routes available to malevolent users. Docker drops all capabilities, except those required by default, using an allowlist rather than a denylist approach. A complete list of possible capabilities can be found in the Linux manpages.

One of the most serious risks associated with using Docker containers is that the container's default set of capabilities and mounts may provide insufficient isolation, either alone or in combination with kernel vulnerabilities.

Docker allows you to add and remove capabilities, allowing you to utilize a profile that is not the default. Docker may become more secure due to feature removal or less secure due to capacity addition. For users, the recommended practice is to disable all capabilities except those explicitly necessary for their operations.

DCT Signature Verification

Only signed images can be run using the Docker Engine, and the Docker Content Trust signature verification is included in the dockerd binary.

The dockerd configuration file specifies this.

Only repositories signed with a user-specified root key can be pulled and run, which can be enabled in daemon.json to enable this capability.

This feature gives administrators more information about enforcing and verifying image signatures than was previously available through the CLI.

Other Kernel Security Features

Capabilities are simply one of several security features that modern Linux kernels provide. With Docker, you may also use current, well-known solutions like TOMOYO, AppArmor, SELinux, GRSEC, etc.

Docker merely enables capabilities at the moment and does not interfere with other systems. This means that hardening a Docker host can be done in various methods. Here are a few examples:

- You can use GRSEC and PAX to run a kernel. Many safety checks are added both at compile and runtime, and many attacks are defeated, thanks to measures like address randomization. It does not need Docker-specific settings because the security features apply to the entire system and not just containers.
- If your distribution includes them, you can utilize security model templates for Docker containers out of the box. These templates offer an extra layer of protection (even though it overlaps greatly with capabilities). For example, we include an AppArmor template, and Red Hat includes SELinux policies for Docker.
- Using your preferred access control technique, you can create your own policies.

Docker Swarm: Overlay Network Encryption and MTLS

When building the overlay network, specify --opt encrypted to encrypt application data to activate IPsec encryption at the vxlan level. Because this encryption has a significant performance penalty, you should test it before deploying it in production.

Docker builds IPsec tunnels across all nodes, where jobs for services tied to the overlay network are scheduled when overlay encryption is enabled. The AES method is used in these tunnels, and the keys are rotated every 12 hours by the manager nodes.

MTLS: Mutual TLS

Nodes use mutual TLS in a swarm to authenticate, authorize, and encrypt node-level interactions. Mutual TLS means that all hosts communicate with other swarm nodes using a certificate.

Docker defines itself as a manager node when you start a swarm from a host. The management node creates a new root certificate authority (CA) and a set of keys (public, private) that are used to encrypt communications with other swarm nodes.

When you add nodes to the swarm, the management node generates two tokens: a worker token and a manager token. The tokens comprise a digest of the root CA certificate and a randomly generated secret. This ensures that the manager knows the request for a node to join the swarm, that it is legitimate, and that the approved node joins the swarm. When a node enters the swarm, the manager supplies it with a certificate that includes a created node ID, which can be used to identify the node under the CN (Common Name) and the role under the OU (organizational unit).

Manage Swarm Security with Public Key Infrastructure (PKI)

Docker's swarm mode public key infrastructure (PKI) architecture makes it simple to securely install a container orchestration system. To authenticate, authorize, and encrypt connections with other nodes in the swarm, nodes in a swarm employ mutual Transport Layer Security (TLS).

Docker marks itself as a management node when you execute docker swarm inits to start a swarm. The management node creates a new root Certificate Authority (CA) and key pair by default, used to encrypt connections with other swarm nodes. You can use the --external-ca flag of the docker swarm init command to specify your own externally-generated root CA.

When you add more nodes to the swarm, the management node generates two tokens: one worker token and one manager token. The root CA's certificate digest and a randomly generated secret are included in each token. The connecting node uses the digest to validate the root CA certificate from the remote manager when it joins the swarm. The remote manager uses the secret to guarantee that the joining node is an approved node.

The manager issues a certificate to each new node that enters the swarm. A randomly generated node ID is included in the certificate to identify the node under the certificate Common Name (CN) and the role under the Organizational Unit (OU). For the duration of the current swarm, the node ID acts as the cryptographically secure node identification.

Encrypting Overlay Networks

It is critical to encrypt communication between nodes in a distributed model like Docker Swarm to prevent possible attackers from accessing sensitive data via network interactions.

The overlay network driver connects many Docker daemon hosts to form a distributed network. When encryption is enabled, this network sits on top of

(overlays) host-specific networks, allowing containers connected to it (including swarm service containers) to communicate securely. Each packet is transparently routed to and from the relevant Docker daemon host and destination container via Docker.

When you create a swarm or attach a Docker host to an existing swarm, the Docker host creates two new networks:

- Ingress is an overlay network that handles the control and data flow for swarm services. If you do not link a swarm service to a user-defined overlay network, it will default to the ingress network
- A docker_gwbridge bridge network that connects each Docker daemon to the other daemons in the swarm.

Docker Security Best Practices

The Docker team retrofitted many modern internet security protocols. Security was not built into networking protocols by the early network inventors.

Security upgrades are often challenging and costly, so implementing security throughout development is a much better technique.

Here are a few choices to consider when it comes to controlling Docker container security:

Avoid Running Containers with Root Privileges

Most Linux administrators are aware of the dangers of giving users full root access, and containers are subject to the same cautions. Thus, creating containers with only the privileges they require is a better approach.

Secure Credentials

Keep your credentials somewhere other than where you use them. Environmental variables are also a better technique to manage container access.

Keeping credentials in the same container is like writing down a password on a sticky note. A breach in one container, in the worst-case scenario, might swiftly propagate throughout an entire application.

Use –PRIVILEGED Tag Sparingly

Containers run in a more secure unprivileged mode by default. Containers will be unable to communicate with other devices. If you need to allow access to other devices on a case-by-case basis, use the –privileged tag.

Use 3rd Party Security Applications

Having a second set of eyes check your security configuration is always a smart

idea. Third-party programs assess your software using the expertise of security experts. They can also assist you in scanning your code for known flaws. Additionally, they frequently feature a user-friendly interface for container security management.

Chapter 09: Docker Enterprise

Introduction

Docker Enterprise Edition (EE), a new version of the Docker platform optimized for business-critical deployments, was introduced in March 2017 by Docker Inc.

Docker Enterprise Edition (EE) is a container platform that is integrated, supported, and certified for:

- CentOS
- Microsoft Windows Server 2016, a server operating system developed by Microsoft
- Oracle Linux, a Linux distribution developed by Oracle

Docker Enterprise Edition (Docker EE) is for enterprise development, and IT teams for building, shipping, and running mission-critical applications in production and at scale. Docker EE is integrated, certified, and maintained to give businesses the industry's most secure container platform.

Docker EE Engine is presently available in two versions:

- If you are solely using Docker EE Engine, use version 18.03.
- If you are using Docker Enterprise Edition 2.0, you will want to use this version (Docker Engine, UCP, and DTR).

Docker EE Basics

Docker Enterprise Engine can be used with Docker EE Basic to manage your container workloads flexibly. Workloads can be managed on Windows, Linux, on-premises, and the cloud.

Enterprise-class support is provided via Docker EE Basic, which includes set SLAs and extended patch maintenance cycles of up to 24 months.

Docker EE Standard and Advanced

Docker EE Standard adds private image management, integrated image signing procedures, and cluster management, including support for Kubernetes and Swarm orchestrators, to the Basic edition.

Docker EE Advanced goes a step further by allowing you to set up node-based RBAC controls, image promotion policies, image mirroring, and vulnerability scanning for your images.

Supported Platforms

On-premises

The table below lists all of the platforms that Docker EE is compatible with. Each of the links in the first column will take you to the platform's installation instructions. Docker EE is a container platform that is integrated, supported, and certified for the cloud providers and operating systems specified.

Capabilities	X86_64 / amd64	IBM Power (ppc64le)	IBM Z (s390x)
CentOS	Yes		
Oracle Linux	Yes		
Red Hat Enterprise Linux	Yes	Yes	Yes
SUSE Linux Enterprise Server	Yes	Yes	Yes
Ubuntu	Yes	Yes	Yes
Microsoft Windows Server 2016	Yes		

Docker Certified Infrastructure

Docker Certified Infrastructure is a prescriptive approach for implementing Docker Enterprise Edition (EE) on various infrastructure platforms. A reference architecture, automation templates, and third-party ecosystem solution briefs are all included in each Docker Certified Infrastructure.

Platform	Docker Enterprise Edition
VMware	Yes
Amazon Web	Yes

Services	
Microsoft Azure	Yes

Docker Enterprise Edition Release Cycles

Each Docker EE release is supported and maintained for a period of 24 months, during which time it receives security and significant bug updates.

The Docker API is unaffected by the Docker platform version. We carefully preserve API backward compatibility and deprecate APIs and functionalities gradually and cautiously. We deprecate features for three stable releases before removing them. Docker 1.13 added dynamic feature negotiation and increased interoperability between clients and servers utilizing different API versions.

Benefit

Docker EE's best feature is certification, which equates to reliability. When businesses run Docker EE on a certified platform, they can be confident that the release is ready for production. That means Docker and the underlying infrastructure partner collaborated to create a stable, tested version that will support and update as needed.

Docker Enterprise comprises the following main components that work together to provide a complete software supply chain, from image generation to secure image storage and deployment.

Docker Engine - Enterprise: The Docker engine for building images and running them in Docker containers that are commercially supported.

Docker Trusted Registry (DTR) - DTR is Docker's enterprise-grade image storage service.

As your consumption grows, DTR is designed to scale horizontally. You can scale DTR to meet your needs and ensure high availability by adding more copies.

Universal Control Plane (UCP)

Docker Universal Control Plane (UCP) provides enterprise-level cluster management. The Universal Control Plane (UCP) manages orchestrators, such as Kubernetes and Swarm, to deploy applications from images. UCP was created with high availability in mind (HA). You can add additional UCP manager nodes to the cluster, and if one fails, another instantly takes its place without affecting

the cluster.

At first glance, UCP may look like “Docker Swarm with a web interface,” but it provides additional features as well, such as:

- Organization and team management
- Role-based access control
- Orchestration with both Docker Swarm and Kubernetes

Docker Trusted Registry (DTR)

Docker Trusted Registry is an on-premises registry that enables businesses to store and manage Docker images on-premises or in their Virtual Private Cloud (VPC) to comply with security and regulatory standards. Universal Control Plane and Trusted Registry are part of the Docker Containers-as-a-Service Platform, an end-to-end IT managed framework that gives developers and IT operations teams agility, portability, and control. They are our on-premises offering for enterprises that need to host their content and management plan on-premises or in a VPC.

Key Benefits

- Granular User Management - SSO with Universal Management Plane, role-based access control, teams/orgs setup, LDAP/AD authentication
- Resource Management - Garbage collection for memory conservation as well as CPU, RAM, and storage monitoring
- Security and Compliance - On-premise deployment, user audit logs, Docker Content Trust image signing

You can install and configure DTR using Launchpad.

DTR is an enhanced, enterprise-ready private Docker registry.

Additional Features

- A web UI
- High availability through multiple registry nodes
- Cache images are geographically close to users
- Role-based access control
- Security vulnerability scanning for images

DTR High Availability

Docker Trusted Registry supports high availability by allowing you to run multiple DTR replicas in a cluster.

A DTR cluster can tolerate failures as long as there is a quorum.

Quorum – more than half of the replicas are available.

DTR Cache

DTR Cache allows you to cache images geographically close to your users, allowing for faster download speeds.

You should know:

- Each cache pulls the image from the main DTR the first time a user requests the images from that cache.
- Users still authenticate using the main DTR URL.
- Users request images from the main DTR URL, and DTR redirects to the cache.
- A user profile setting determines which cache a user will pull images from.
- This means authentication and image pulling are completely transparent to users; they do not need to think about caches.

Easy to Use and Manage

Great tools are useless if they are challenging to use. With a one-click install and GUI-based system configuration, Trusted Registry quickly gets you up and running. Non-disruptive updates also make it simple to install the most recent patches and releases, guaranteeing a secure and up-to-date environment. Upgrading to the most recent minor, patch, or major release is as simple as clicking a single button within the application. Admins can also monitor system health directly via the web interface.

Secure Access and Control

You may use Trusted Registry to manage who has access to Docker images and what kind of access they have. When users use Trusted Registry through the LDAP/AD integration option, they will authenticate directly against your organization's directory services. Configure several role-based access levels for your users and groups of users in organizations, such as admin, user, or read-only rights. Create organizations to organize people into teams and give them access to repos. Admins can use Docker Content Trust to sign images after they have been created for enhanced security and to ensure that the most recent version of an image is being used.

The user audit logs are likewise stored in the Trusted Registry, allowing you to keep track of all user activity in the system.

Chapter 10: Docker Kubernetes Service

Introduction

Kubernetes is a cutting-edge orchestration solution for deploying, scaling and managing distributed applications that have swept the industry in recent years. However, due to its intrinsic complexity, only a small number of businesses have been able to fully reap the benefits of Kubernetes, with 96 percent of enterprise IT groups unable to handle it on their own. At Docker, you understand that a large part of Kubernetes' perceived complexity originates from a lack of clear security and manageability configurations that most organizations expect and demand from production-grade software.

Docker Kubernetes Service allows you to orchestrate container workloads in UCP interfaces with the flexibility of a Kubernetes cluster.

Kubernetes Orchestration in Docker

Orchestrator Type

Each UCP node has an Orchestrator type, which determines whether the node will run workloads managed by Docker Swarm or Kubernetes.

Docker Swarm

The node will run Docker Swarm-managed workloads.

Kubernetes

The node will run Kubernetes-managed workloads.

Mixed

The node will run workloads managed by both Docker Swarm and Kubernetes.

Namespaces

Every Kubernetes object exists in a namespace. A namespace allows you to organize your objects and separate them between different teams or applications.

Application Configuration in Kubernetes

Application Configuration

Application Configuration is all about managing configuration data and passing data to applications.

Kubernetes uses ConfigMaps and Secrets to store configuration data and pass it to containers.

ConfigMaps

ConfigMaps allows you to store configuration data in a key/value format. This data can include simple values or larger data blocks such as configuration files.

Secrets

Secrets are similar to ConfigMaps, but their data is encrypted at rest. Use secrets to store sensitive data, such as passwords or API tokens.

Using Config Data

Configuration data stored in ConfigMaps or Secrets can be passed to our containers in two ways:

Environment Variables

The configuration values can be passed in as environment variables visible to the **container** process at runtime.

Volume Mounts

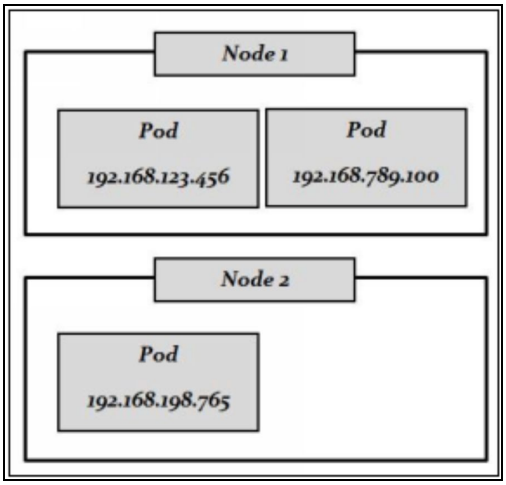
Configuration data can be mounted to the container file system, which will appear in the form of files.

The Kubernetes Network Model

The Kubernetes Network Model is a set of standards that define how networking between pods behaves.

Regardless of whatever node a pod is executing on, the Kubernetes Network Model describes how pods communicate with one another.

Each pod in the cluster has its IP address.



Services and DNS

Service allows us to expose an application running as a set of pods.

A service exposes a set of pods, load-balancing client traffic across them. These pods can then be dynamically created and destroyed without interruption to clients using the service.

Service Types

Each service has a type. The type determines the behavior of the service and what it can be used for.

ClusterIP (the default)

Exposes to pods within the cluster using a unique IP on the clusters' virtual network.

NodePort

Exposes the service outside the cluster on each Kubernetes node, using a static port.

LoadBalancer

It uses a cloud providers' load balancing functionality to expose the service to the outside world.

ExternalName

Expose an external resource inside the cluster network using DNS.

Cluster DNS

Kubernetes includes a cluster DNS that helps containers locate services using domain names.

Pods within the same namespace service can use the service name as a domain name.

Pods in a different namespace must use the entire domain name, which takes the form: my-svc.my-namespace.svc.cluster-domain.example.

Deployments

Deployments allow you to manage changes to a set of replica pods. They specify the desired state for a set of pods and work to achieve and maintain that state, even if it is changed.

You can use deployments to do things like rolling updates and scaling.

DaemonSets

DaemonSets are a tool that allows you to run a replica pod dynamically on each

node in the cluster.

They will run a replica on each node and create a new replica on new nodes as they are added.

Scheduling

In Kubernetes, scheduling refers to the process of selecting a node on which to run a workload.

The Kubernetes Scheduler handles scheduling.

Node Taints

Node taints control which pods will be allowed to run on which nodes. Pods can have tolerations, which override taints for the specific pod.

Each taint has an effect.

The execute effect does two things to pods that do not have the appropriate toleration:

- It prevents the scheduling of new pods on the nodes
- Evicts existing pods

Resource Requests

In the container spec, you can specify resource requests for resources such as memory and CPU.

The schedule will avoid scheduling pods on nodes that do not have enough resources to satisfy the requests.

Probes

Probes allow you to customize how the Kubernetes cluster determines the state of each container.

Liveness Probes

Liveness Probes check whether the container is healthy.

Readiness Probes

Readiness Probes check whether the container is ready to service user requests.

Storage with Volumes

Volumes

Kubernetes uses volumes to provide simple storage to containers. Volume storage is more persistent than the container file system and can remain even if a container is restarted or re-created.

Volumes are part of the pod spec, and the same volume can be mounted to multiple containers.

Volume Types

A volume type determines how the underlying storage is handled. There are many volume types available.

nfs

Shared network storage is provided using nfs.

hostPath

Files are stored in a directory on the worker node. The same files will not appear if the pod runs on another node.

emptyDir

Temporary storage that is deleted if the pod is deleted. It helps share simple storage between containers in a pod.

Storage with PersistentVolumes

PersistentVolumes

PersistentVolumes allows you to manage storage resources abstractly. They let you define a set of available storage resources as a Kubernetes object, then later claim those storage resources for use in your pods.

A PersistentVolumesClaim defines a request for storage resource that can be mounted to a pod's containers.

Reclaim Policy

A PersistentVolume has a reclaim policy that determines what happens when the associated claims are deleted.

Retain

It keeps the volume and its data and allows manual reclaiming. Admin is responsible for cleaning up existing data.

Delete

Deletes both the PersistentVolume and its underlying storage infrastructure (such as a cloud storage object).

Recycle

Deletes the data contained in the volumes and allows it to be used again.

Expanding A PersistentVolumes

You can expand PVC by simply editing the object to have a larger size.

For this to work, the storage class must support volume expansion. If this is the case, the storage class's allowVlomeExpansion field will be set to true.